

Лабораторная работа №11

Численное дифференцирование

Дифференцирование с помощью сплайнов

*Procedure Spline3(N: integer; X,Y: mas; S0,SN: real;
Var A,B,C,D: mas);*

Определяет коэффициенты сплайна, которые будут использованы процедурой *DifSpline* для получения численных значений производных в заданной точке.

В х о д н ы е п а р а м е т р ы :

N – число узлов сплайна;

X: mas – массив аргументов;

Y: mas – массив значений функции;

S0 – значение абсциссы начальной точки сплайна;

SN – значение абсциссы конечной точки сплайна;

В ы х о д н ы е п а р а м е т р ы :

A,B,C,D: mas – массивы коэффициентов сплайна.

Массивы имеют тип *mas = array[0..n+1] of real*.

Схема процедуры *Spline3* показана на рисунке 36.

Текст процедуры:

```
procedure Spline3(N:integer;X,Y:mas;S0,SN:real; Var A,B,C,D:mas);
var F:mas;
    H2,H3,p:real;
    i,j:integer;
begin
    fillchar(a,sizeof(a),0);
    fillchar(b,sizeof(a),0);
    fillchar(c,sizeof(a),0);
    fillchar(d,sizeof(a),0);
    fillchar(f,sizeof(a),0);
    H2:=X[2]-X[1];
    H3:=X[3]-X[2];
    A[1]:=(2*(H2+H3))/H3;
    f[1]:=(6/h3)*(((y[3]-y[2])/h3)-((y[2]-y[1])/h2))-(h2*s0)/h3;
    for i:=4 to n-1 do begin
        h2:=x[i-1]-x[i-2];
        h3:=x[i]-x[i-1];
        a[i-2]:=(2/h3)*(h2+h3);
        b[i-2]:=h2/h3;
        f[i-2]:=(6/h3)*(((y[i]-y[i-1])/h3)-((y[i-1]-y[i-2])/h2)));
```

```

end;
h2:=x[n-1]-x[n-2];
h3:=x[n]-x[n-1];
p:=2*(h2+h3);
b[1]:=h2/p;
f[n-2]:=(6/p)*(((y[n]-y[n-1])/h3)-((y[n-1]-y[n-2])/h2))-(h3*sn)/p;
d[1]:=1/a[1]; c[1]:=f[1];
For i:=2 to n-3 do begin
  d[i]:=1/(a[i]-b[i]*d[i-1]); c[i]:=f[i]-b[i]*d[i-1]*c[i-1];
end;
d[n-2]:=(f[n-2]-b[1]*d[n-3]*c[n-3])/(1-b[1]*d[n-3]);
for i:=n-3 downto 1 do d[i]:=d[i]*(c[i]-d[i+1]);
c[1]:=s0; c[n]:=sn;
For i:=2 to n-1 do c[i]:=d[i-1];
For i:=1 to n do begin

a[i]:=0; b[i]:=0; d[i]:=0;
end;
For i:=2 to n do begin
  h2:=x[i]-x[i-1]; d[i]:=(c[i]-c[i-1])/h2;
  b[i]:=h2*c[i]/2-(Sqr(h2))*d[i]/6+(y[i]-y[i-1])/h2;
  a[i]:=y[i];
end;
//z:=f;
end;

```

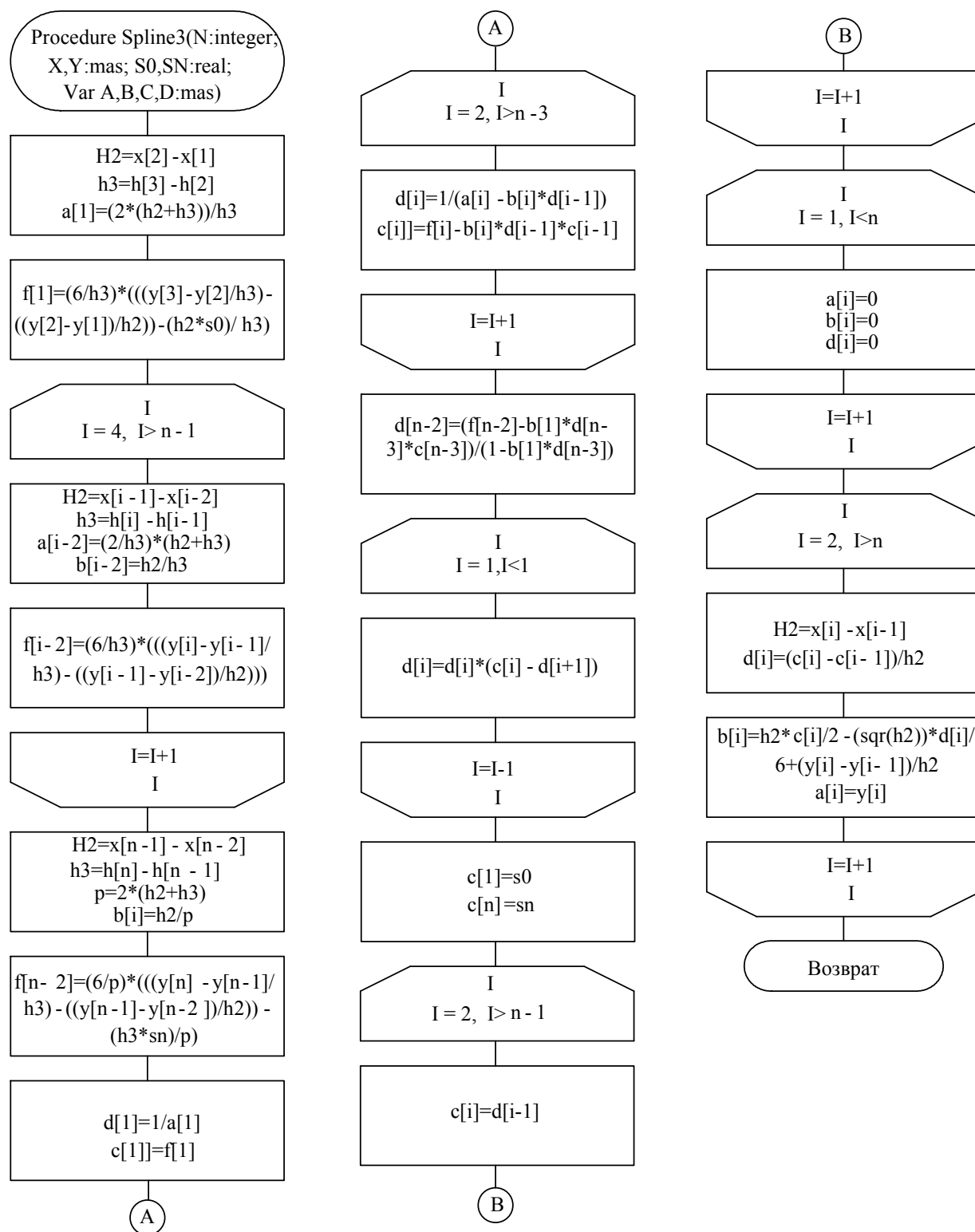


Рисунок 36 – Схема алгоритма вычисления коэффициентов сплайна

*Procedure DifSline(x,y,z:mas; xx:real; n:integer;
var s1,s2:real; error:boolean);*

Процедура на основе данных, полученных от процедуры *Spline3*, вычисляет значение производной в заданной точке.

В х о д н ы е п а р а м е т р ы :

n: integer – число узлов сплайна;

xx: real – абсцисса точки, для которой необходимо вычислить производную;

x: integer – массив аргументов;

y: mas – массив значений функции;

z: mas – массив коэффициентов сплайна.

В ы х о д н ы е п а р а м е т р ы :

error: boolean – переменная будет иметь значение "истина", если точка *xx* находится за пределами интервала заданных значений аргументов.

Все массивы имеют тип *mas=array[0..n+1] of real*.

Схема процедуры *DifSline* показана на рисунке 37.

Текст процедуры:

```
procedure DifSline(x,y,z:mas; xx:real; n:integer; var s1,s2:real; error:boolean);
var i,j,k:integer;
    t,t1,t2,t12:real;
    hi:real;
begin
    i:=1;
    while (xx >= x[i])and(i<=n) do inc(i);
    if i>n then begin
        error:=true; exit;
    end;
    hi:=x[i]-x[i-1];
    t:=(xx-x[i-1])/hi; //(x[i]-x[i-1]);
    t1:=1-t;
    t2:=t*t;
    t12:=t1*t1;
    s1:=-6*t1*t*(y[i-1]-y[i])/hi;
    s1:=s1+t1*z[i-1]*(1-3*t)+z[i]*(3*t-2)*t;
    s2:=6*(y[i-1]-y[i])*(2*t-1)/hi;
    s2:=s2+(z[i-1])*(6*t-4)-z[i]*(2-6*t);
    s2:=s2/hi;
end;
```

Варианты заданий приведены в таблице 9.

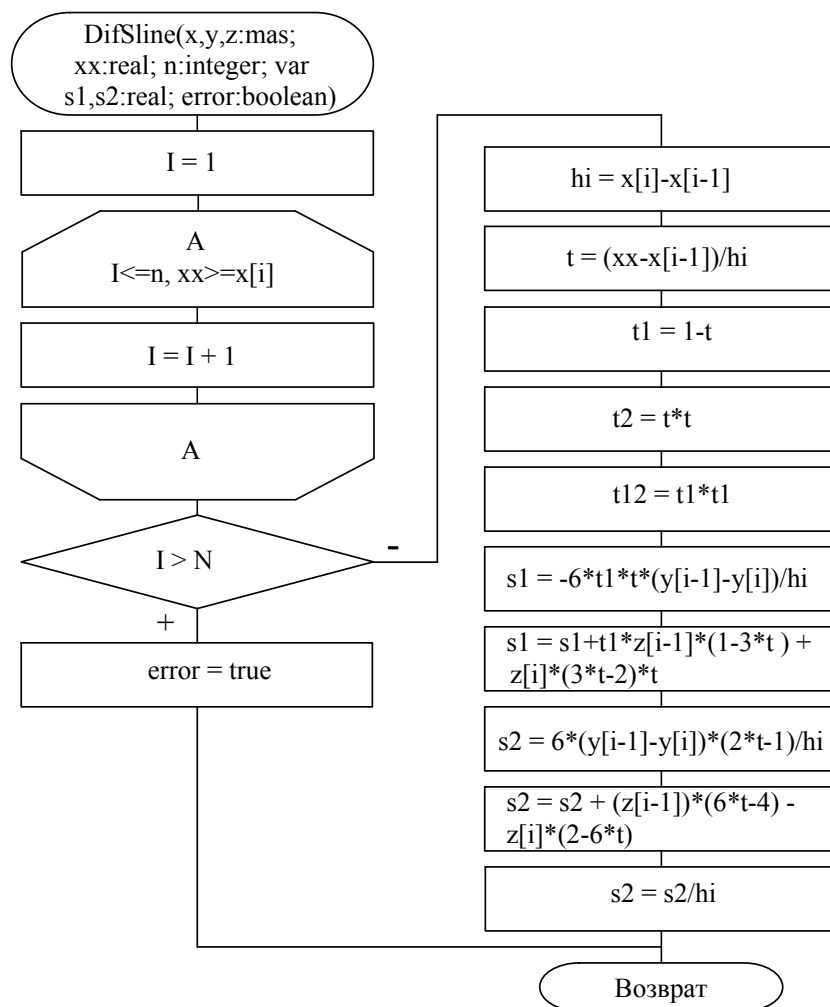
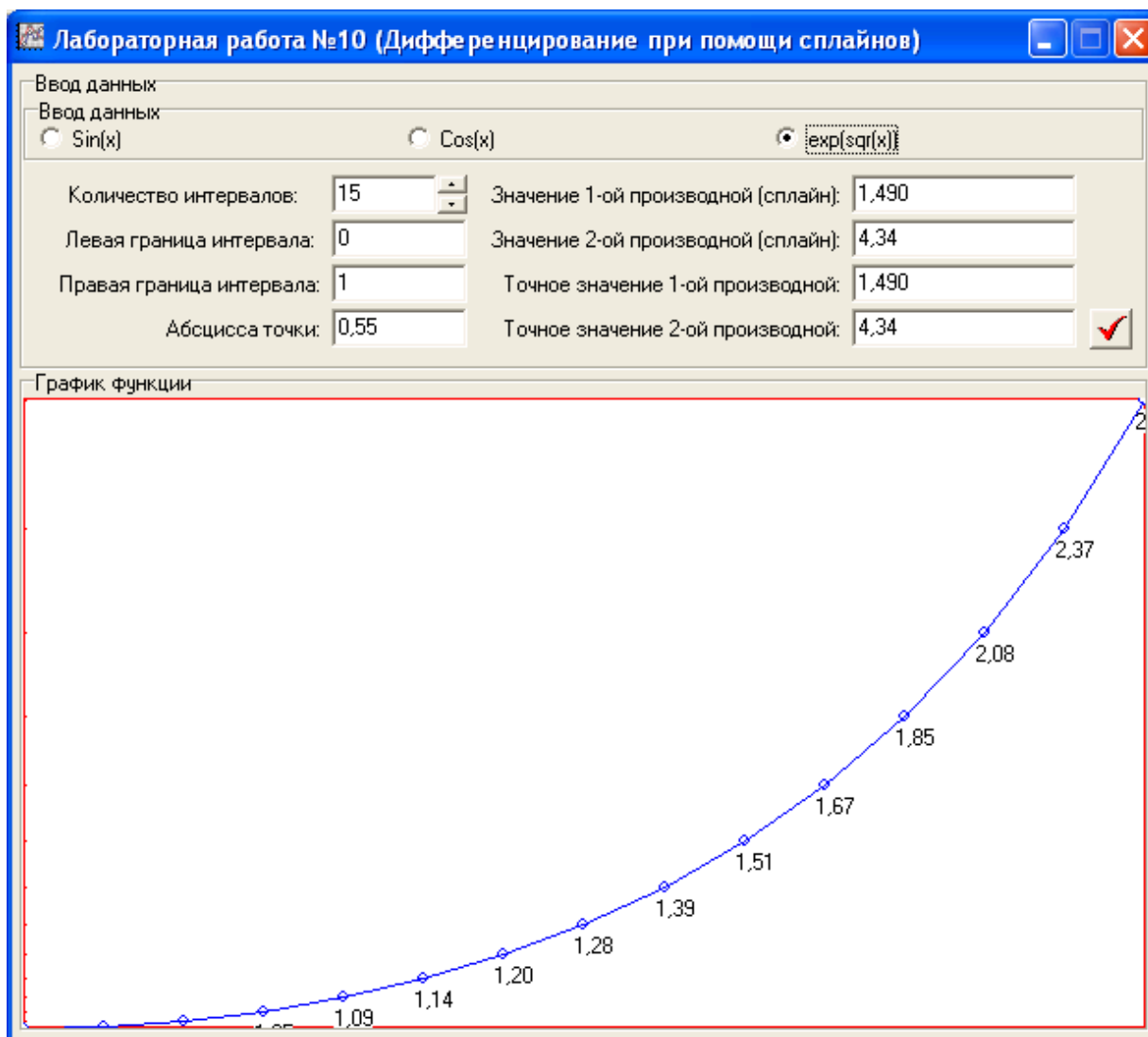


Рисунок 37 – Схема алгоритма дифференцирования с помощью сплайнов

Пример. Найти производные функции e^{x^2} в точке $x=0,55$. Функция задана таблично в 15 узлах.

Вычисления по программе привели к следующим результатам:



Текст программы:

```
program Lab10VM;

uses
  Forms,
  Unit1 in 'Unit1.pas' {Form1};

{$R *.res}

begin
  Application.Initialize;
  Application.Title := 'Лабораторная работа №10';
  Application.CreateForm(TForm1, Form1);
  Application.Run;
end.
```

```

unit Unit1;

interface

uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
  Dialogs, ExtCtrls, Grids, StdCtrls, ComCtrls, Buttons;

const n = 300;
type
  mas = array[0..n+1] of real;
  TForm1 = class(TForm)
    GroupBox1: TGroupBox;
    SpeedButton1: TSpeedButton;
    UpDown1: TUpDown;
    LabeledEdit1: TLabeledEdit;
    LabeledEdit2: TLabeledEdit;
    LabeledEdit3: TLabeledEdit;
    LabeledEdit4: TLabeledEdit;
    LabeledEdit5: TLabeledEdit;
    LabeledEdit6: TLabeledEdit;
    GroupBox2: TGroupBox;
    Image1: TImage;
    LabeledEdit7: TLabeledEdit;
    RadioGroup1: TRadioGroup;
    LabeledEdit8: TLabeledEdit;
    procedure ClearImage;
    procedure FormCreate(Sender: TObject);

    procedure SpeedButton1Click(Sender: TObject);
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;
  x,ys:mas;
  xc:real;
  yc,m:integer;
  dx,dt:real;
  a,b,max,min,koefx,koefy:real;
implementation

{$R *.dfm}
function f(x:real):real;
begin
  case Form1.RadioGroup1.ItemIndex of
    0: Result:=sin(x);
    1: Result:=cos(x);
    2: Result:=exp(sqr(x));
  end;
end;

```

```

else Result:=0;
end;
end;

function fx(x:real):real;
begin
  case Form1.RadioGroup1.ItemIndex of
    0: Result:=cos(x);
    1: Result:=-sin(x);
    2: Result:=2*x*exp(sqr(x));
    else Result:=0;
  end;
end;

function fxx(x:real):real;
begin
  case Form1.RadioGroup1.ItemIndex of
    0: Result:=-sin(x);
    1: Result:=-cos(x);
    2: Result:=2*exp(sqr(x))*(1+2*x*x);
    else Result:=0;
  end;
end;

procedure GetParams1;
begin
  // xc:=Form1.Image1.Width div 2;
  yc:=Form1.Image1.Height div 2;
end;

procedure GetParams2;
var i:integer;
begin
  m:=strtoint(Form1.LabeledEdit1.Text);
  a:=strtofloat(Form1.LabeledEdit2.Text);
  b:=strtofloat(Form1.LabeledEdit3.Text);
  if a<0 then xc:=(-a)*koefx else xc:=-a*koefx;
  dx:=(b-a)/(m-1);
  x[0]:=a-dx; ys[0]:=f(x[0]);
  x[m+1]:=b+dx; ys[m+1]:=f(x[m+1]);
  x[1]:=a; ys[1]:=f(a);
  x[m]:=b; ys[m]:=f(b);
  for i:=2 to m do begin
    x[i]:=x[i-1]+dx;
    ys[i]:=f(x[i]);
  end;
  max:=f(a); min:=f(a);
  for i:=1 to m do begin
    if max<f(x[i]) then max:=f(x[i]);
    if min>f(x[i]) then min:=f(x[i]);
  end;
  dt:=(Form1.Image1.Width-1)/(b-a);

```

```

    koefy:=(Form1.Image1.Height-1)/(max - min);
    koefx:=(Form1.Image1.Width-1)/(b-a);
end;

procedure TForm1.ClearImage;
var i:integer;
begin
    with Image1.Canvas do begin
        Brush.Color:=clWhite;
        Pen.Color:=clRed;
        FillRect(Image1.Canvas.ClipRect);
        Rectangle(Image1.Canvas.ClipRect);
        Pen.Width:=1;
    end;
end;

procedure ClearImage2;
var x0,y0:integer;
    x1,y:integer;
    i:integer;
begin
    Form1.ClearImage;
    //y0:=round(Form1.Image1.Height-koefy*(min));
    y0:=round(Form1.Image1.Height-1-(Form1.Image1.Height-1)*(min/(min-max)));
    Form1.Image1.Canvas.MoveTo(0,y0);
    Form1.Image1.Canvas.LineTo(Form1.Image1.Width-1,y0);
    for i:=1 to m do begin
        x1:=round(i*dx*koefx);
        Form1.Image1.Canvas.MoveTo(x1,y0-2);
        Form1.Image1.Canvas.LineTo(x1,y0+2);
        //Form1.Image1.Canvas.TextOut(x1,y0-2,floattostrf(x[i],ffFixed,3,3));
    end;
    x0:=round((a*(Form1.Image1.Width-1)/(a-b)));
    Form1.Image1.Canvas.MoveTo(x0,0);
    Form1.Image1.Canvas.LineTo(x0,Form1.Image1.Height-1);
    for i:=1 to m do begin
        Form1.Image1.Canvas.MoveTo(x0-2,round((Form1.Image1.Height)-(ys[i]-
min)*koefy));
        Form1.Image1.Canvas.LineTo(x0+2,round((Form1.Image1.Height)-(ys[i]-
min)*koefy));
    end;
end;

procedure Spline3(N:integer;X,Y:mas;S0,SN:real; Var A,B,C,D:mas);
var F:mas;
    H2,H3,p:real;
    i,j:integer;
begin
    fillchar(a,sizeof(a),0);
    fillchar(b,sizeof(a),0);
    fillchar(c,sizeof(a),0);
    fillchar(d,sizeof(a),0);

```

```

fillchar(f,sizeof(a),0);
H2:=X[2]-X[1];
H3:=X[3]-X[2];
A[1]:=(2*(H2+H3))/H3;
f[1]:=(6/h3)*(((y[3]-y[2])/h3)-((y[2]-y[1])/h2))-(h2*s0)/h3;
for i:=4 to n-1 do begin
    h2:=x[i-1]-x[i-2];
    h3:=x[i]-x[i-1];
    a[i-2]:=(2/h3)*(h2+h3);
    b[i-2]:=h2/h3;
    f[i-2]:=(6/h3)*(((y[i]-y[i-1])/h3)-((y[i-1]-y[i-2])/h2)));
end;
h2:=x[n-1]-x[n-2];
h3:=x[n]-x[n-1];
p:=2*(h2+h3);
b[1]:=h2/p;
f[n-2]:=(6/p)*(((y[n]-y[n-1])/h3)-((y[n-1]-y[n-2])/h2))-(h3*sn)/p;
d[1]:=1/a[1]; c[1]:=f[1];
For i:=2 to n-3 do begin
    d[i]:=1/(a[i]-b[i]*d[i-1]); c[i]:=f[i]-b[i]*d[i-1]*c[i-1];
end;
d[n-2]:=(f[n-2]-b[1]*d[n-3]*c[n-3])/(1-b[1]*d[n-3]);
for i:=n-3 downto 1 do d[i]:=d[i]*(c[i]-d[i+1]);
c[1]:=s0; c[n]:=sn;
For i:=2 to n-1 do c[i]:=d[i-1];
For i:=1 to n do begin
    a[i]:=0; b[i]:=0; d[i]:=0;
end;
For i:=2 to n do begin
    h2:=x[i]-x[i-1]; d[i]:=(c[i]-c[i-1])/h2;
    b[i]:=h2*c[i]/2-(Sqr(h2))*d[i]/6+(y[i]-y[i-1])/h2;
    a[i]:=y[i];
end;
//z:=f;
end;

```

```

procedure DifSline(x,y,z:mas; xx:real; n:integer; var s1,s2:real; error:boolean);
var i,j,k:integer;
    t,t1,t2,t12:real;
    hi:real;
begin
    i:=1;
    while (xx >= x[i])and(i<=n) do inc(i);
    if i>n then begin
        error:=true; exit;
    end;
    hi:=x[i]-x[i-1];
    t:=(xx-x[i-1])/hi; //(x[i]-x[i-1]);
    t1:=1-t;
    t2:=t*t;
    t12:=t1*t1;
    s1:=-6*t1*t*(y[i-1]-y[i])/hi;

```

```

    s1:=s1+t1*z[i-1]*(1-3*t)+z[i]*(3*t-2)*t;
    s2:=6*(y[i-1]-y[i])*(2*t-1)/hi;
    s2:=s2+(z[i-1])*(6*t-4)-z[i]*(2-6*t);
    s2:=s2/hi;
end;

procedure DrawPoints(l:boolean);
var i:byte;
begin
    Form1.Image1.Canvas.Pen.Color:=clBlue;
    form1.Image1.Canvas.MoveTo(round(x[1]*koefx-xc),Form1.Image1.Height-
round((ys[1]-min)*koefy));
    for i:=1 to m do begin
        form1.Image1.Canvas.Ellipse(round((x[i])*koefx+xc)-3,
            Form1.Image1.Height-round((ys[i]-min)*koefy)-3,
            round((x[i])*koefx+xc)+3,
            Form1.Image1.Height-round((ys[i]-min)*koefy)+3);
        form1.Image1.Canvas.TextOut(round((x[i])*koefx+xc-5),
            Form1.Image1.Height-round((ys[i]-min)*koefy)+5,
            floattostrf(ys[i],ffFixed,3,2));
    end;
end;

procedure TForm1.FormCreate(Sender: TObject);
begin
    ClearImage;
end;

procedure TForm1.SpeedButton1Click(Sender: TObject);
var i:integer;
    z,s,c,d,z2:mas;
    xx,s1,s2:real;
    err:boolean;
begin
    GetParams2;
    ClearImage2;
    err:=false;
    xx:=strtofloat(LabeledEdit6.Text);
    Spline3(m,x,ys,x[0],x[m+1],z,s,c,d);
    DifSline(x,ys,s,xx,m,s1,s2,err);
    if err then showmessage('!!!');
    LabeledEdit4.Text:=floattostrf(s1,ffFixed,3,3);
    LabeledEdit5.Text:=floattostrf(s2,ffFixed,3,2);
    LabeledEdit7.Text:=floattostrf(fx(xx),ffFixed,3,3);
    LabeledEdit8.Text:=floattostrf(fxx(xx),ffFixed,3,2);
    DrawPoints(true);
    form1.Image1.Canvas.MoveTo(round(x[1]*koefx+xc),
        Form1.Image1.Height-round((ys[1]-min)*koefy));
    for i:=2 to m do begin
        a:=x[i]; b:=x[i+1];
        while a<=b do begin
            form1.Image1.Canvas.LineTo(round(a*koefx+xc),

```

```

        Image1.Height-round((z[i]+s[i]*(a-x[i])
        +(c[i]/2)*(a-x[i])*(a-x[i])
        +(d[i]/6)*(a-x[i])*(a-x[i])*(a-x[i]-min)*koefy));
    a:=a+dt;
end;
end;
end;
end.

```

Лабораторная работа № 12

Численное интегрирование

Пусть требуется найти определенный интеграл

$$I = \int_a^b f(x)dx,$$

где функция $f(x)$ непрерывна на отрезке $[a; b]$.

Для приближенного вычисления интегралов чаще всего подынтегральную функцию заменяют «близкой» ей вспомогательной функцией, интеграл от которой вычисляется аналитически. За приближенное значение интеграла принимают значение интеграла от вспомогательной функции.

Заменим функцию на отрезке $[a; b]$ ее значением в середине отрезка. Искомый интеграл, равный площади криволинейной фигуры, заменяется на площадь прямоугольника. Из геометрических соображений нетрудно записать *формулу прямоугольников*

$$\int_a^b f(x) \approx f\left(\frac{a+b}{2}\right)(b-a).$$

Приблизив $f(x)$ линейной функцией и вычислив площадь соответствующей трапеции, получим *формулу трапеций*

$$\int_a^b f(x) \approx \frac{1}{2}(f(a) + f(b))(b-a).$$

Если же приблизить подынтегральную функцию параболой, проходящей через точки $(a, f(a))$, $\left(\frac{a+b}{2}, f\left(\frac{a+b}{2}\right)\right)$, $(b, f(b))$, то по-

лучим *формулу Симпсона*

$$\int_a^b f(x) \approx \frac{1}{6}\left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right)(b-a)$$